AWS Cloud Project Documentation

High-Availability & Secure Two-Tier Web Architecture on AWS

Project Name	High-Availability Two-Tier Web Architecture on AWS
Domain	AWS Cloud / DevOps Infrastructure
Author	Shaikh Altaf (AWS Account ID: 569378723978)
Date	14-10-2026

1. Project Title

Project Name: Highly Available & Secure Two-Tier Web Architecture on AWS

Domain: AWS Cloud / DevOps

Author: Shaikh Altaf

Date: 14-10-2026

2. Project Overview

This project implements an enterprise-grade, highly available, and secure two-tier web application architecture on Amazon Web Services (AWS). It establishes a logically isolated custom Virtual Private Cloud (VPC) spanning two Availability Zones (us-east-1a and us-east-1b) in the us-east-1 (N. Virginia) region to eliminate single points of failure.

Incoming user traffic for the registered domain (twotire.shop) resolves through Amazon Route 53 with SSL/TLS termination via AWS Certificate Manager (ACM). An internet-facing Application Load Balancer (ALB) balances HTTP/HTTPS traffic across Amazon EC2 instances managed by an Auto Scaling Group. In the backend, an Amazon Aurora MySQL database cluster operates within private subnets with internet ingress disabled, ensuring strict data protection and defense-in-depth.

3. Objectives

- **Understand AWS Services:** Understand and architect core AWS networking, compute, database, and security services.
- **Step-by-Step Implementation:** Deploy custom VPC, multi-AZ subnets, routing, NAT Gateways, EC2 compute instances, ALB, and Aurora MySQL database clusters.
- **Security Best Practices:** Enforce least privilege, security group chaining, private subnet database isolation, and ACM SSL encryption.
- **Monitor & Troubleshoot:** Verify system health with ALB target group checks, instance status checks, and multi-node application responses.

Edit with WPS Office

4. Problem Statement

Deploying web applications on single-server or single-AZ infrastructures introduces severe reliability and security risks. A hardware disruption in a single data center causes total downtime. Furthermore, placing databases in public-facing subnets exposes mission-critical records to unauthorized attacks and data theft. This project resolves these issues by implementing a modular two-tier architecture that isolates backend databases in private subnets, auto-scales compute resources across multiple availability zones, and automatically routes traffic around unhealthy instances.

5. Architecture

The architecture is structured into a presentation/web tier and an isolated data tier distributed across two availability zones (us-east-1a and us-east-1b):

Edit with WPS Office

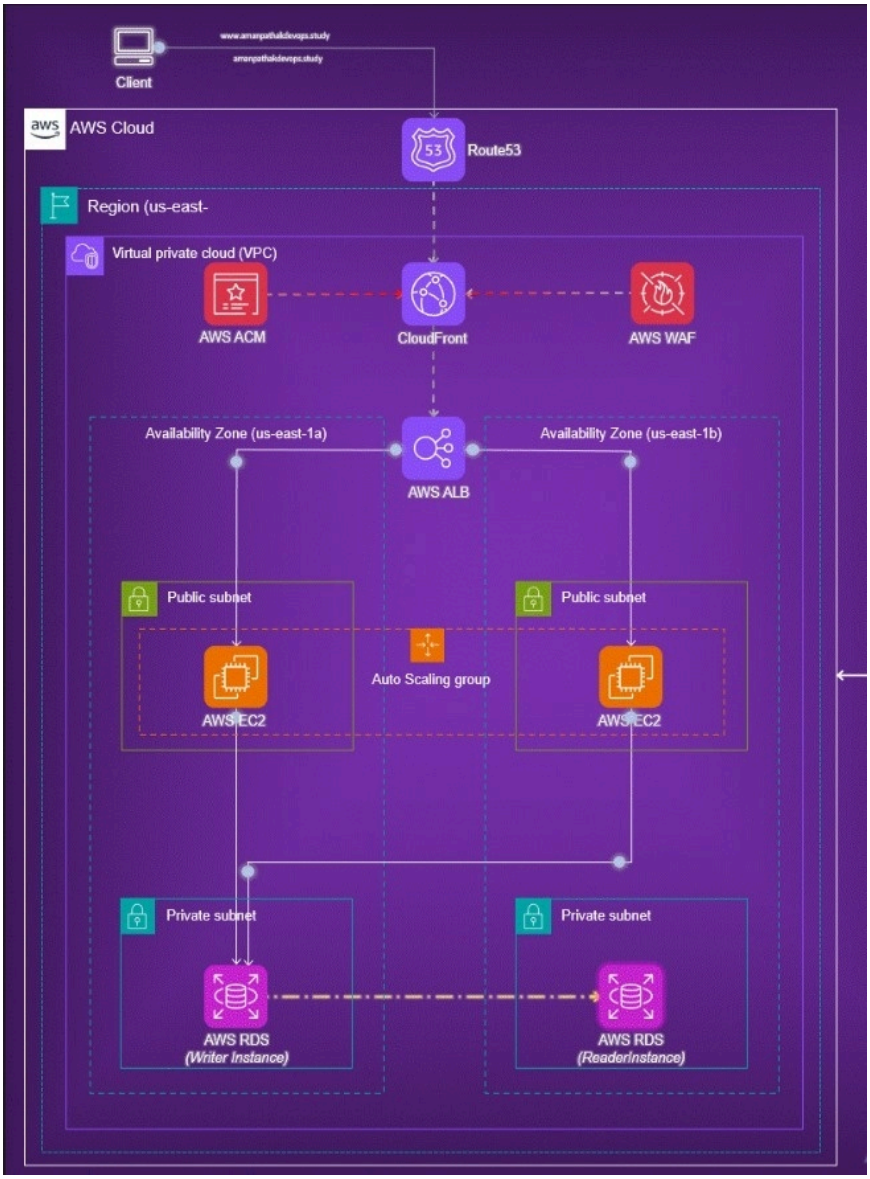


Figure 1: High-Availability & Secure Two-Tier AWS Cloud Architecture

Components Used

- **Compute:** Amazon EC2 (t3.micro instances: app-ec2, app-tire2, data-ec2), EC2 Auto Scaling Group (myec2-auto-scale), Launch Template (mytemp), Custom AMIs (app-image, data-image, my-image).
- **Storage & Database:** Amazon Aurora MySQL Regional Cluster (app-database: Writer in us-east-1b, Reader in us-east-1a, db.r7g.large), EBS General Purpose SSDs.

Edit with WPS Office

- **Networking:** Custom VPC (Two_tire_project-vpc), 4 Multi-AZ Subnets (2 Public, 2 Private), Route Tables, Internet Gateway (Two_tire_project-igw), NAT Gateway (mynat), Application Load Balancer (MY-ALB), Target Group (mytg).
- **Security & Identity:** AWS Certificate Manager (ACM SSL/TLS for twotire.shop), Granular Security Groups (web-sg, app-sg, autoscale-sg, data-sg).
- **Monitoring:** ALB Target Group Health Checks, EC2 3/3 Status Checks, CloudWatch Metrics.
- **DNS & Domain:** Amazon Route 53 (twotire.shop hosted zone with ALB Alias routing), GoDaddy Custom Nameserver delegation.

6. Prerequisites

- **AWS Account:** AWS Account with active administrative credentials (Account ID: 569378723978).
- **IAM Permissions:** IAM permissions for VPC, EC2, ELB, Auto Scaling, RDS, ACM, and Route 53.
- **Linux & DevOps Skills:** Linux bash navigation, Apache/Nginx web server setup, HTML deployment.
- **Domain Registrar:** Domain name twotire.shop registered with DNS management capabilities.
- **Tools:** AWS Management Console, SSH Client / EC2 Instance Connect, Web Browser.

7. Implementation Steps

Step 1: Create Custom Virtual Private Cloud (VPC)

Purpose: Provides a logically isolated virtual network to deploy and govern the two-tier application infrastructure securely.

Procedure: Navigate to the VPC console in us-east-1. Click 'Create VPC', assign Name tag 'Two_tire_project-vpc', specify IPv4 CIDR block, and enable DNS resolution and hostnames.

Expected Result: VPC is provisioned with ID vpc-035be38234590783e in Available state.

Evidence: VPC console showing Two_tire_project-vpc (vpc-035be38234590783e).

Edit with WPS Office

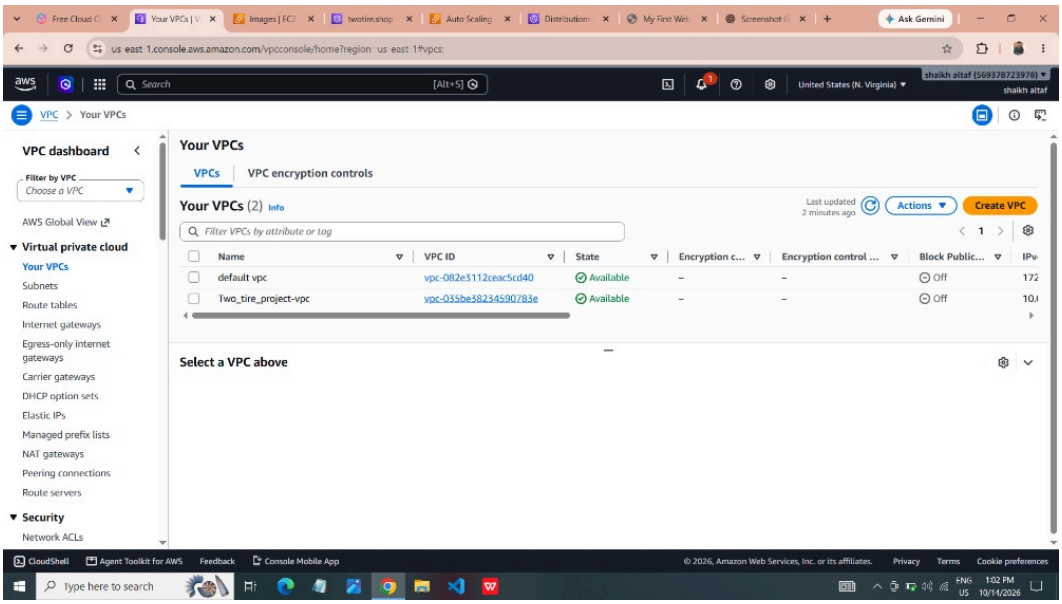


Figure 2: Custom VPC Two_tire_project-vpc Created and Available in us-east-1

Step 2: Provision Multi-AZ Public and Private Subnets

Purpose: Segregates public web ingress from private data storage and distributes workloads across us-east-1a and us-east-1b for high availability.

Procedure: Create two public subnets (Two_tire_project-subnet-public1-us-east-1a, Two_tire_project-subnet-public2-us-east-1b) and two private subnets (Two_tire_project-subnet-private1-us-east-1a, Two_tire_project-subnet-private2-us-east-1b). Enable auto-assign public IPv4 addresses for public subnets.

Expected Result: All 4 subnets are created successfully across the two availability zones in Available state.

Evidence: Subnet console listing all 4 custom subnets.

Edit with WPS Office

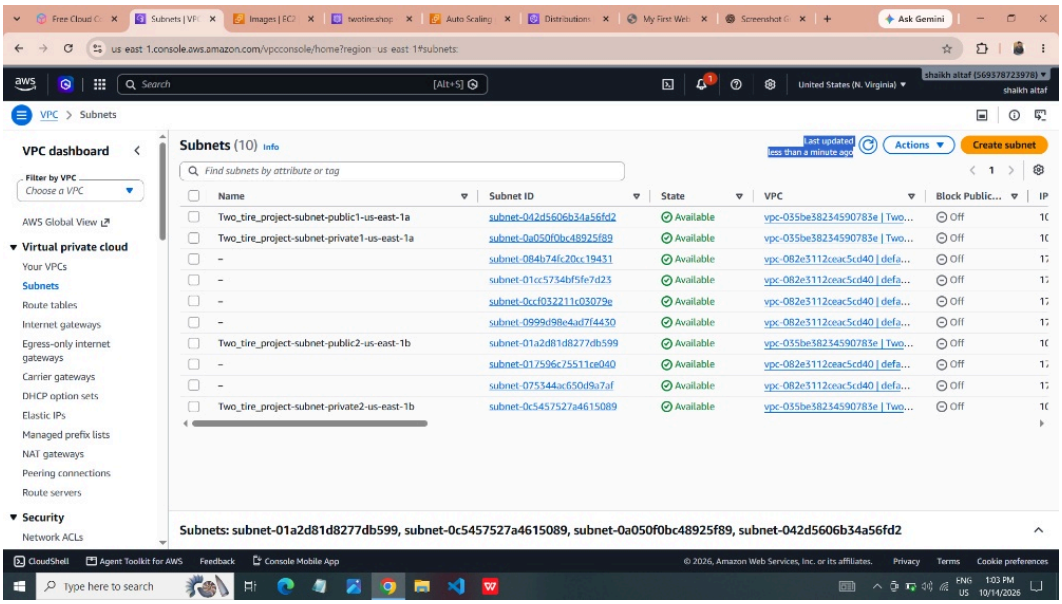


Figure 3: Multi-AZ Public and Private Subnets Configured in us-east-1a and us-east-1b

Step 3: Attach Internet Gateway (IGW)

Purpose: Enables two-way communication between instances in the public subnets and the internet.

Procedure: Create Internet Gateway 'Two_tire_project-igw' (igw-02373043f5f142ad4) and attach it to Two_tire_project-vpc.

Expected Result: Internet Gateway status displays 'Attached'.

Evidence: Internet Gateway console confirmation.

Edit with WPS Office

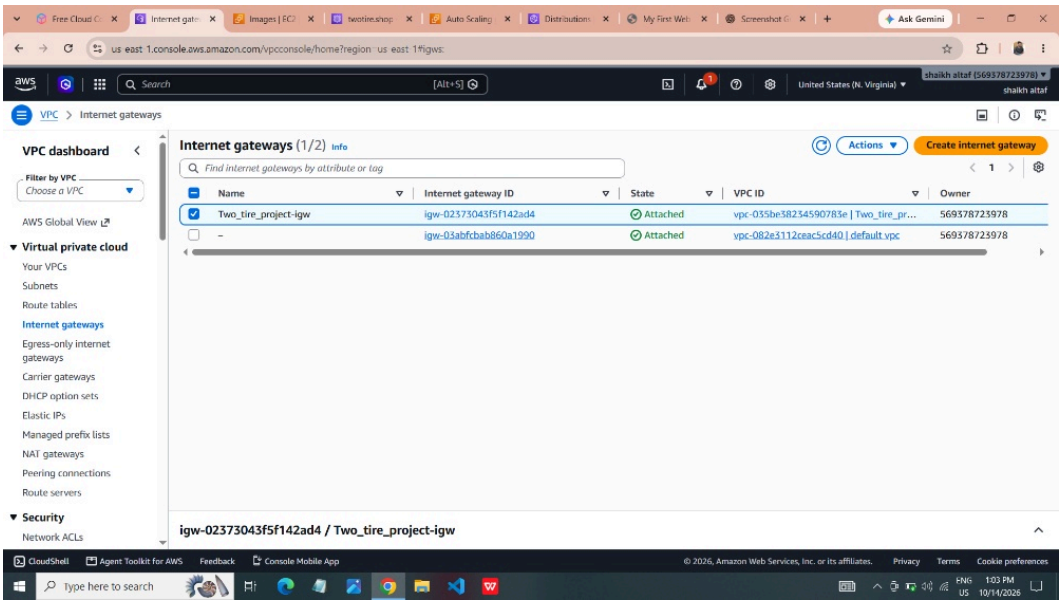


Figure 4: Internet Gateway Attached to Custom VPC

Step 4: Provision Managed NAT Gateway

Purpose: Enables instances in private subnets to reach external repositories for updates without exposing them to incoming internet connections.

Procedure: Create NAT Gateway 'mynat' (nat-128dcdcb5d4f7152) with Connectivity type Public. Place it inside public subnet Two_tire_project-subnet-public1-us-east-1a and allocate an Elastic IP.

Expected Result: NAT Gateway transitions to Available state.

Evidence: NAT Gateway console verification.

Edit with WPS Office

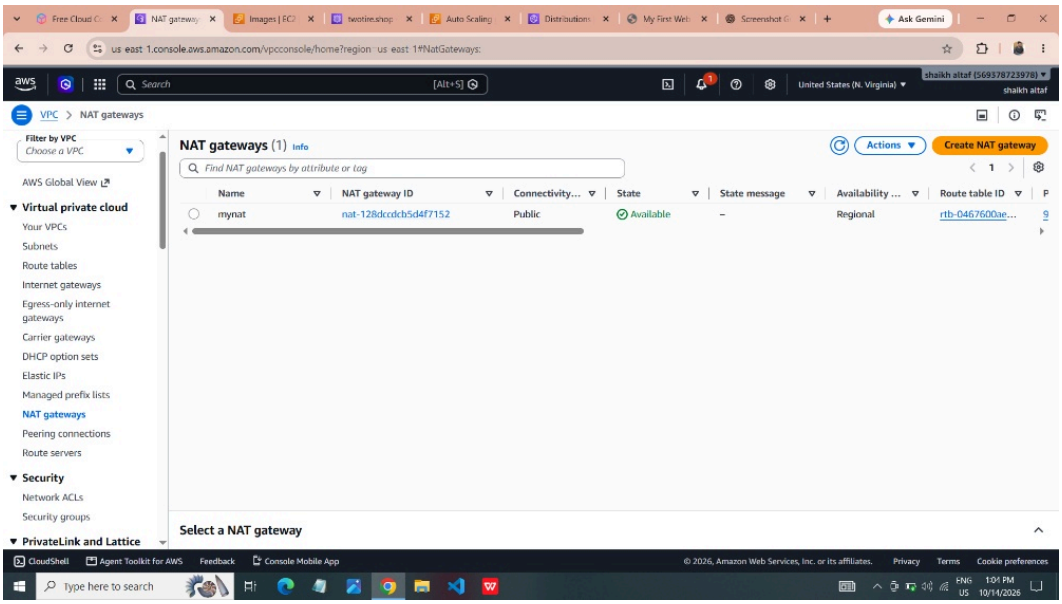


Figure 5: Managed NAT Gateway mynat Active in Public Subnet

Step 5: Configure Custom Route Tables

Purpose: Directs internet-bound traffic from public subnets to the IGW and from private subnets to the NAT Gateway.

Procedure: Create public route table Two_tire_project-rtb-public and associate both public subnets with destination 0.0.0.0/0 pointing to Two_tire_project-igw. Create private route tables pointing 0.0.0.0/0 to mynat.

Expected Result: Explicit subnet associations confirmed across all route tables.

Evidence: Route tables console showing public and private route tables.

Edit with WPS Office

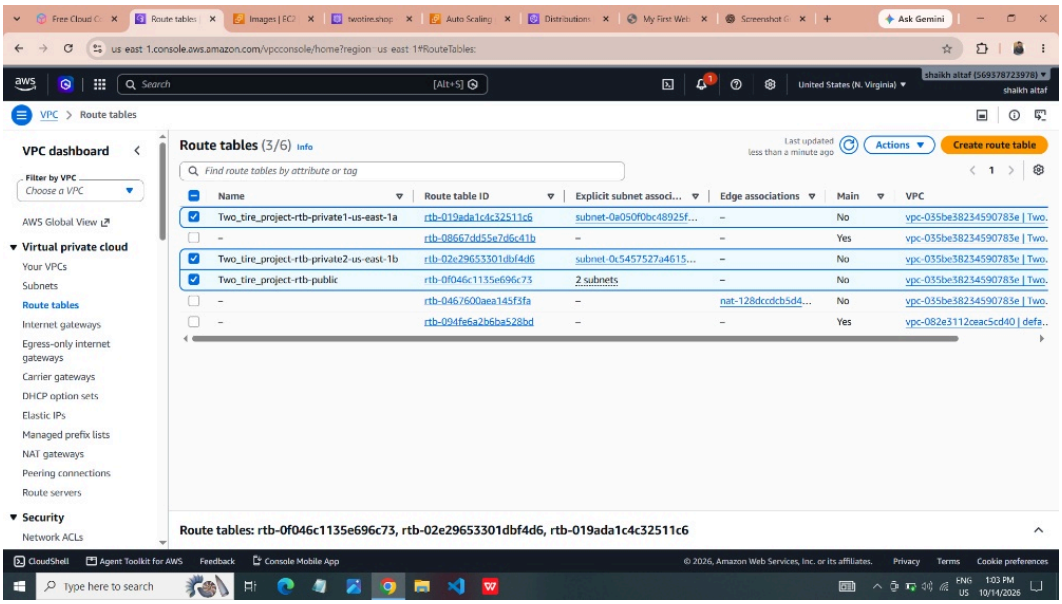


Figure 6: Custom Route Tables Configured with IGW and NAT Gateway Routes

Step 6: Define Tiered Security Groups

Purpose: Enforces virtual firewall security at the instance level following least privilege.

Procedure: Create web-sg (sg-0da7503d39ed91e78) allowing HTTP/HTTPS (80/443), app-sg (sg-05781c62481637a1c) allowing traffic only from ALB, autoscale-sg (sg-039d45366aec6b874), and data-sg (sg-0104fb9c066b59bcb) allowing MySQL 3306 only from app-sg.

Expected Result: All 4 tiered security groups provisioned within Two_tire_project-vpc.

Evidence: Security Groups console verification.

Edit with WPS Office

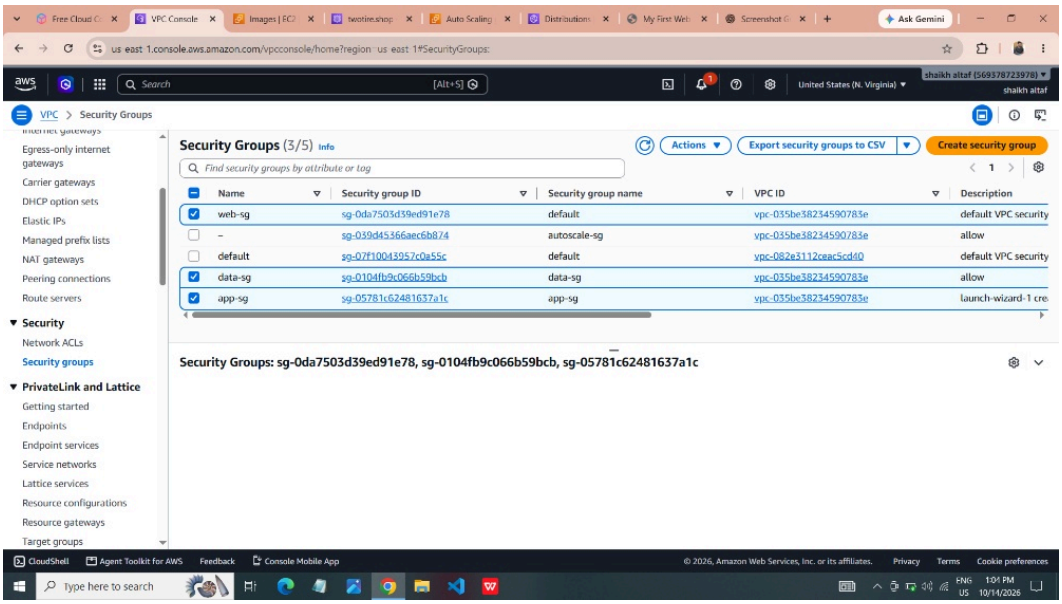


Figure 7: Tiered Security Groups (web-sg, app-sg, autoscale-sg, data-sg)

Step 7: Launch Compute and Data EC2 Instances

Purpose: Deploys web application instances in public subnets and backend data instances in private subnets across multiple AZs.

Procedure: Launch app-ec2 (i-04b5ca3ff52bb9299, t3.micro) in us-east-1a and app-tire2 (i-03e382a4dc0f77f14, t3.micro) in us-east-1b. Launch private data instances in private subnets.

Expected Result: All 4 instances reach Running state with 3/3 checks passed.

Evidence: EC2 Instances console showing running instances across AZs.

Edit with WPS Office

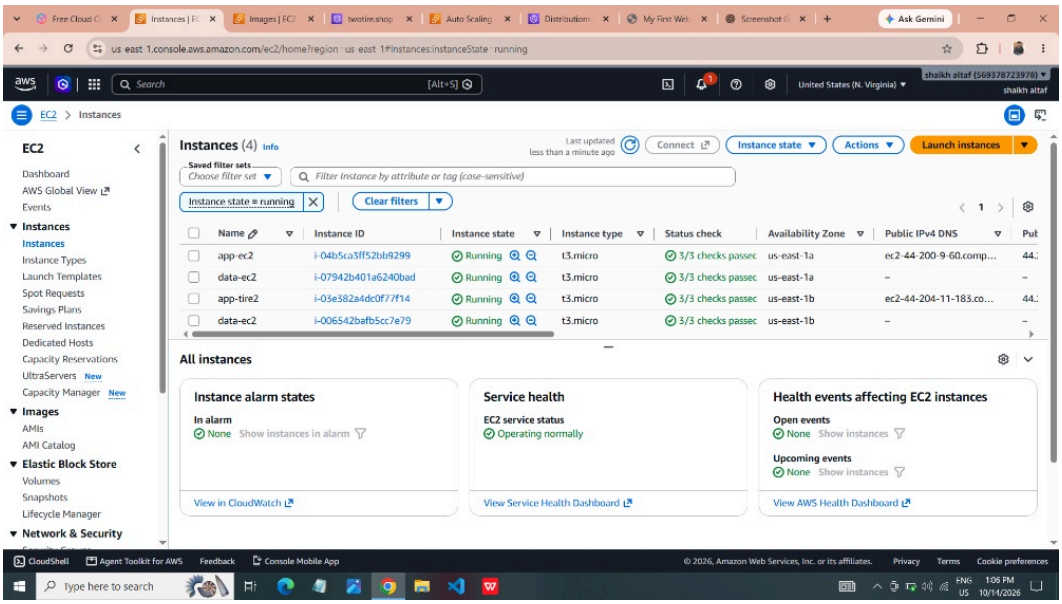


Figure 8: EC2 Compute and Data Instances Running Across Availability Zones

Step 8: Configure Target Group & Application Load Balancer

Purpose: Distributes incoming user traffic evenly across healthy compute instances in multiple availability zones.

Procedure: Create Target Group 'mytg' (HTTP:80) and register app-ec2 and app-tire2. Deploy internet-facing Application Load Balancer 'MY-ALB' across public subnets in us-east-1a and us-east-1b with an HTTP listener forwarding to mytg.

Expected Result: ALB is Active and target group shows 2 Healthy targets.

Evidence: Target group health status and ALB details console.

Edit with WPS Office

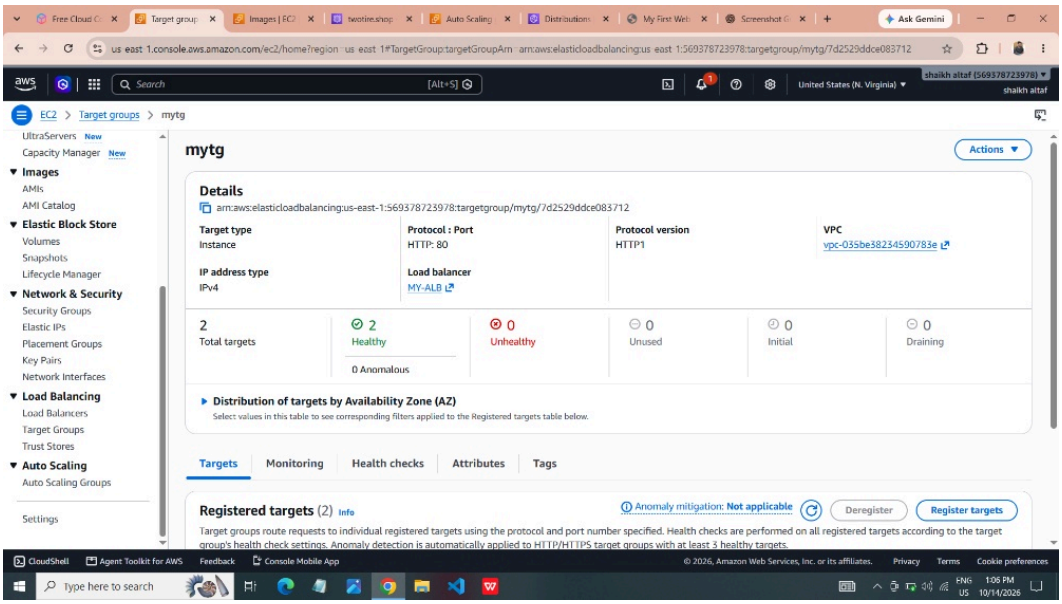


Figure 9: Target Group mytg Confirming 2 Healthy Targets

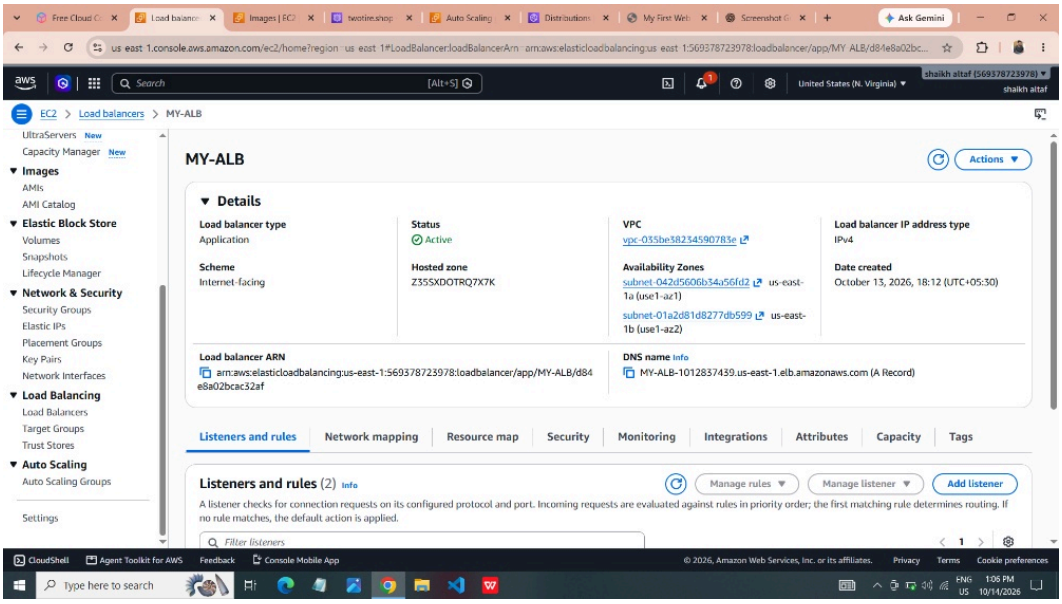


Figure 10: Application Load Balancer MY-ALB Active with Multi-AZ Mapping

Step 9: Create Custom Amazon Machine Images (AMIs)

Purpose: Captures fully configured, tested application and database instances into reusable machine images for automated deployment.

Procedure: Select running instance app-ec2, click Actions > Image and templates > Create

Edit with WPS Office

image, and name it 'app-image' (ami-008998d7d3a3fbda0). Similarly create 'data-image' and 'my -image'.

Expected Result: AMIs created and available under Private AMIs.

Evidence: EC2 AMIs console listing app-image, data-image, and my-image.

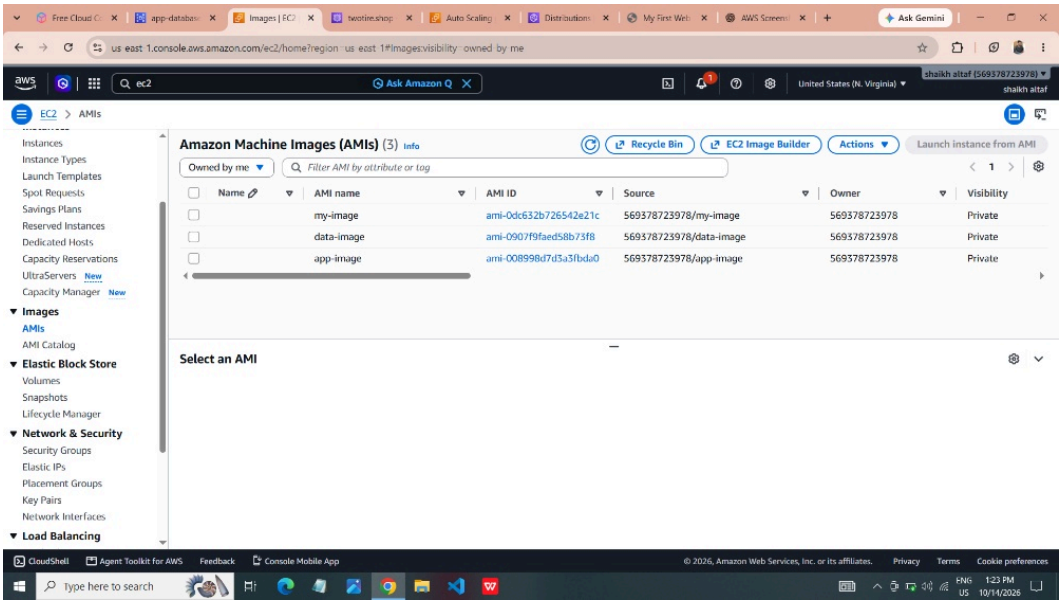


Figure 11: Custom Machine Images (AMIs) Created for Scaling

Step 10: Configure EC2 Launch Template

Purpose: Standardizes instance launch specifications (AMI, instance type, security groups, user data) for Auto Scaling.

Procedure: Create launch template 'mytemp' (lt-0c098e0e59b55c886). Select AMI app-image, instance type t3.micro, and security group autoscale-sg.

Expected Result: Launch template created with Default version 1.

Evidence: EC2 Launch Templates console.

Edit with WPS Office

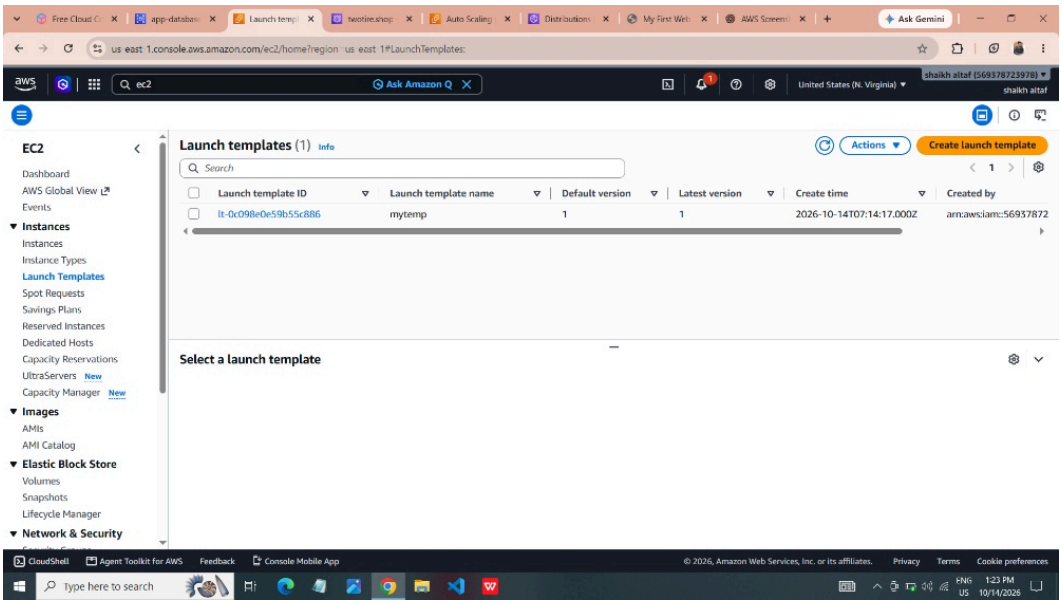


Figure 12: Launch Template mytemp Configured with Custom AMI

Step 11: Configure Auto Scaling Group (ASG)

Purpose: Dynamically scales EC2 compute instances based on traffic demand and automatically replaces degraded instances.

Procedure: Create Auto Scaling Group 'myec2-auto-scale' associated with Launch Template mytemp and target group mytg. Set scaling limits (Min: 0, Desired: 0, Max: 4) across public subnets in us-east-1a and us-east-1b.

Expected Result: ASG created with status 'At desired capacity'.

Evidence: Auto Scaling Groups console overview.

Edit with WPS Office

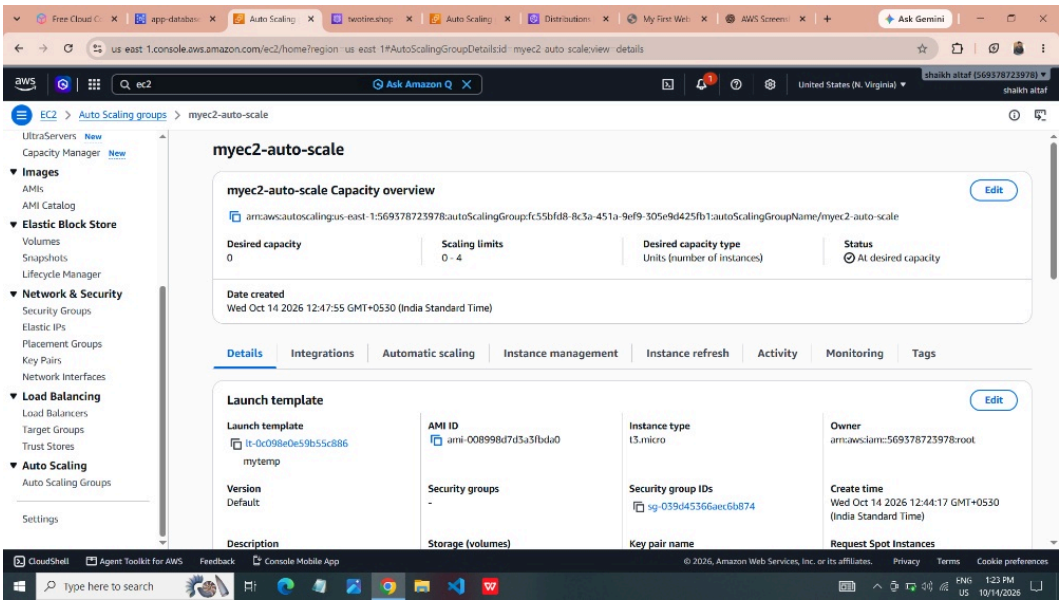


Figure 13: Auto Scaling Group myec2-auto-scale Configured with Launch Template mytemp

Step 12: Deploy Amazon Aurora MySQL Multi-AZ Cluster

Purpose: Provides managed, enterprise-grade relational database storage with automated multi-AZ replication and failover.

Procedure: In Amazon RDS, create database cluster 'app-database' with Aurora MySQL engine. Provision Writer instance 'app-database-instance-1' (db.r7g.large) in us-east-1b and Reader instance 'app-database-instance-1-reader' in us-east-1a. Disable public access.

Expected Result: Aurora cluster and both DB instances transition to Available state.

Evidence: Amazon RDS Aurora console showing cluster and instances.

Edit with WPS Office

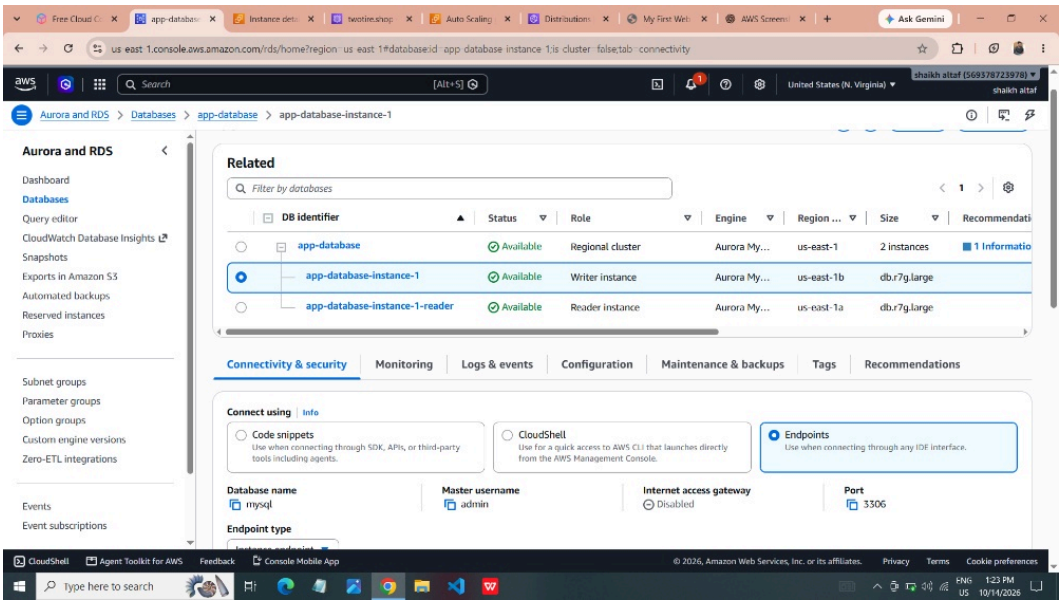


Figure 14: Amazon Aurora MySQL Regional Cluster with Writer and Reader Instances

Step 13: Request & Validate SSL/TLS Certificate in ACM

Purpose: Secures web traffic with in-transit encryption (HTTPS) across the registered domain.

Procedure: Request a public SSL certificate in AWS Certificate Manager for twotire.shop using DNS validation. Add the validation CNAME record to Route 53.

Expected Result: Certificate status changes to 'Issued' with Success validation.

Evidence: ACM console showing Issued certificate for twotire.shop.

Edit with WPS Office

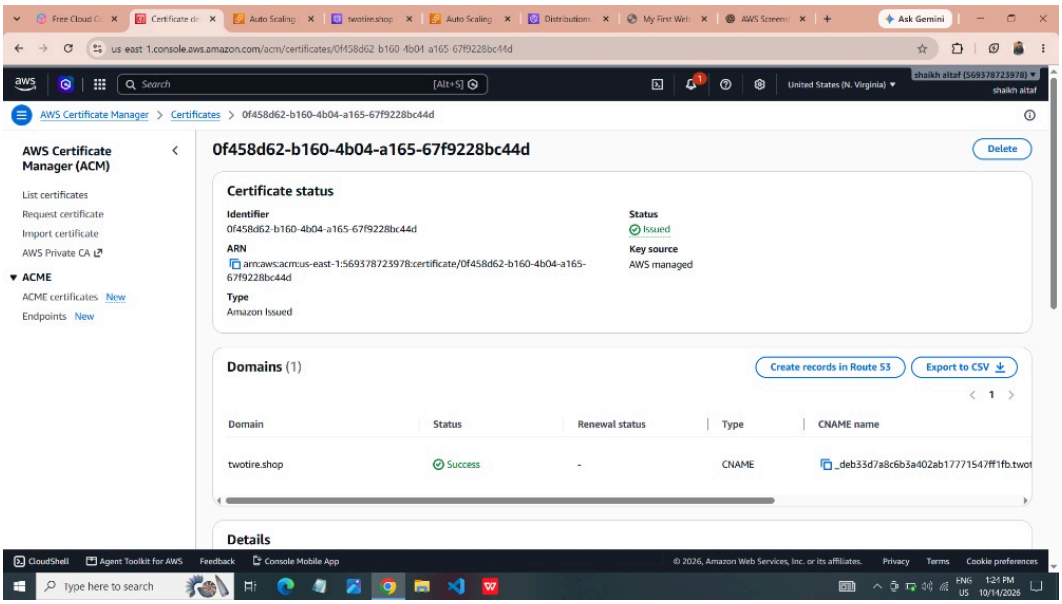


Figure 15: AWS Certificate Manager SSL/TLS Certificate Issued for twotire.shop

Step 14: Configure Route 53 DNS & Delegate GoDaddy Nameservers

Purpose: Directs domain traffic from twotire.shop to the Application Load Balancer and verifies domain ownership.

Procedure: Create a public hosted zone in Route 53 for twotire.shop. Add an A record Alias pointing to MY-ALB. Update GoDaddy custom nameservers to point to the four assigned AWS Route 53 name servers.

Expected Result: Route 53 hosted zone active and GoDaddy domain delegation completed.

Evidence: Route 53 hosted zone and GoDaddy nameserver settings screenshots.

Edit with WPS Office

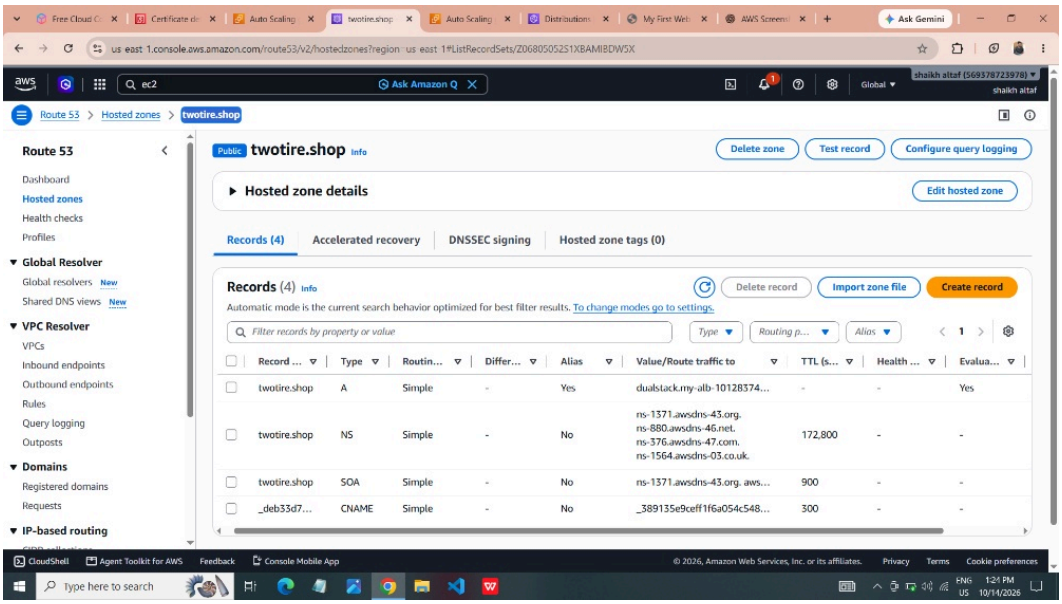


Figure 16: Route 53 Hosted Zone Records for twotire.shop with ALB Alias

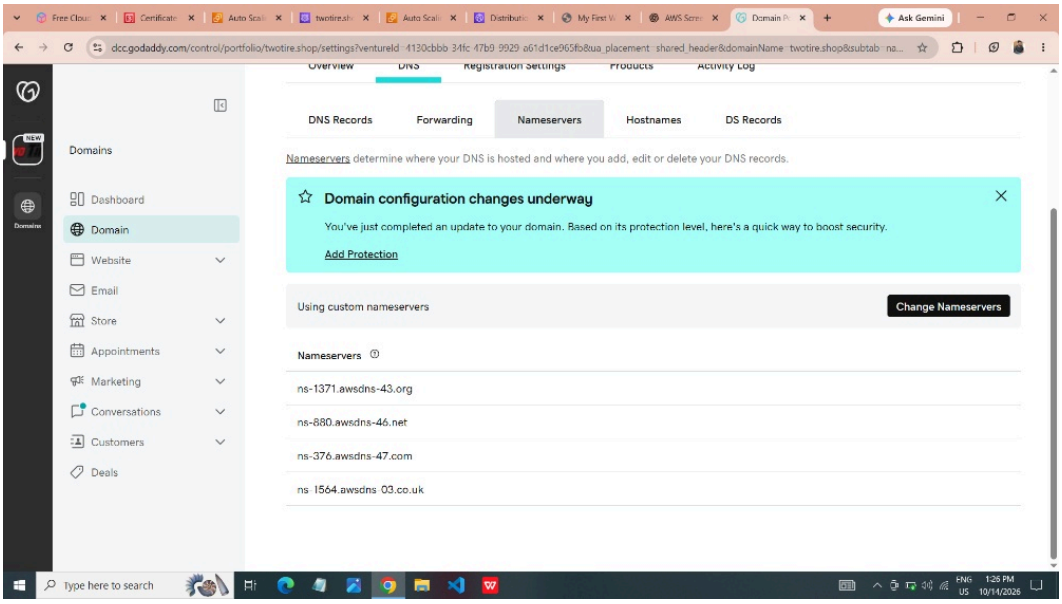


Figure 17: GoDaddy Domain Management Delegated to AWS Route 53 Nameservers

8. Security Configuration

- **Network Segmentation:** Private subnets host Aurora MySQL database instances with zero internet ingress routes. Outbound software updates flow through NAT Gateway mynat.
- **Security Group Chaining:** Security groups are tightly chained: ALB allows 80/443; app-sg

Edit with WPS Office

permits HTTP only from ALB; data-sg permits port 3306 only from app-sg.

- **Database Access Restriction:** Amazon Aurora MySQL configured with 'Internet access gateway: Disabled', ensuring data is completely unreachable from public networks.
- **In-Transit Encryption:** Managed SSL/TLS encryption via AWS Certificate Manager protects client-to-server traffic.

9. Testing and Validation

Comprehensive verification was performed across all network, load balancing, DNS, and application layers:

Test	Expected Result	Actual Result	Status
VPC & Multi-AZ Subnets	4 subnets active across us-east-1a and us-east-1b	All 4 subnets active and properly associated	Passed
ALB Target Health	Registered EC2 targets pass HTTP health checks	2 of 2 instances reporting Healthy in mytg	Passed
Aurora DB Multi-AZ	Writer in us-east-1b, Reader in us-east-1a active	Cluster and both instances reporting Available	Passed
DNS & Domain Routing	twotire.shop resolves to ALB endpoint	GoDaddy delegated to Route 53; A record active	Passed
SSL/TLS Certificate	ACM certificate validated via DNS	Certificate Issued with Success domain status	Passed
Load Balancing Node 1	Browser request displays application version 1	twotire.shop displays 'TECHNOVA.' web application	Passed
Load Balancing Node 2	Subsequent request balances to second node	twotire.shop displays 'Nova.' web application	Passed

Application Live Verification Evidence

Accessing the live domain twotire.shop confirmed seamless load balancing across multi-AZ compute targets:

Edit with WPS Office

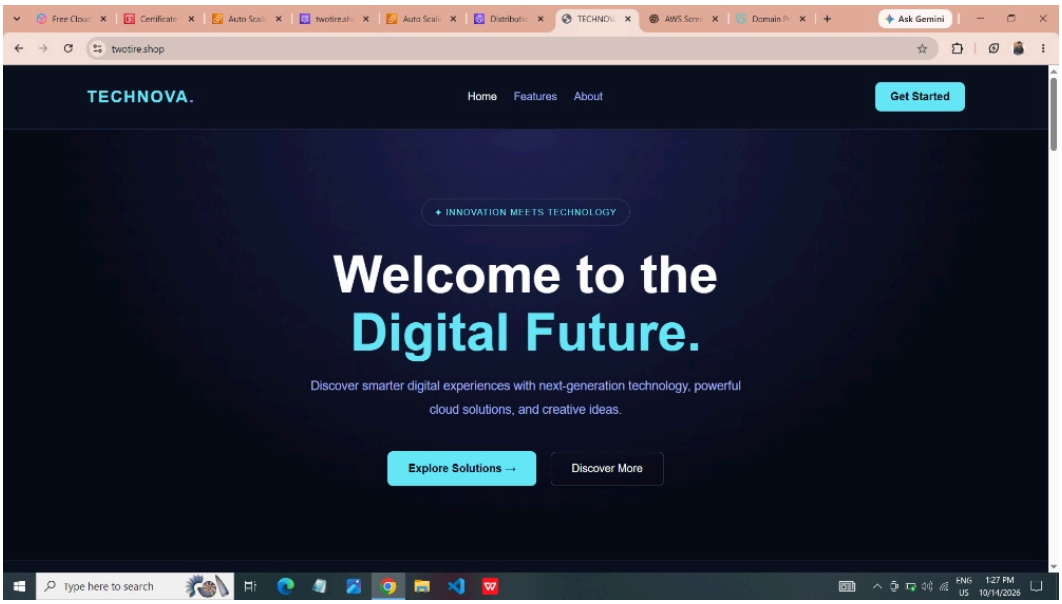


Figure 18: Live Application Serving 'TECHNOVA.' via twotire.shop

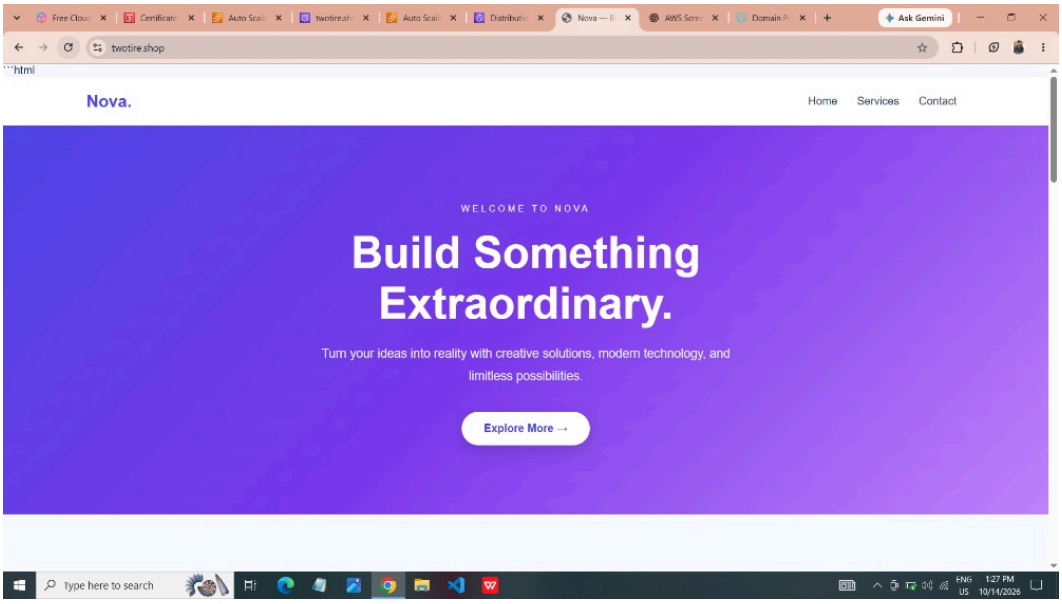


Figure 19: Live Application Serving 'Nova.' Demonstrating Balanced Traffic Delivery

10. Troubleshooting

Issue Observed	Root Cause	Solution Implemented

Edit with WPS Office

Issue Observed	Root Cause	Solution Implemented
Target group instances show Unhealthy	Security group app-sg did not allow port 80 from ALB or web server was stopped.	Configured app-sg inbound rule to accept HTTP from web-sg and verified web server daemon status.
Domain twotire.shop not resolving	Nameserver propagation delay after delegating from GoDaddy to Route 53.	Verified custom nameservers matched Route 53 NS delegation and flushed local DNS cache.
ACM certificate stuck in Pending	DNS validation CNAME record was not added to the Route 53 hosted zone.	Used ACM 'Create records in Route 53' button to automatically insert the required validation CNAME.
Private instances unable to update packages	Missing default route targetting NAT Gateway in private route tables.	Added route 0.0.0.0/0 pointing to mynat in both private route tables.

11. Cost Management

- **Active AWS Resources Created:** Amazon EC2 (t3.micro), Auto Scaling Group, ALB (MY-ALB), Aurora MySQL (app-database, 2x db.r7g.large), NAT Gateway (mynat), Route 53 Hosted Zone.
- **Cost Optimization & Estimates:** Primary billable components are Aurora db.r7g.large instances and the NAT Gateway. Utilizing t3.micro keeps compute within free tier allowances.
- **Teardown / Cleanup Strategy:** Delete Auto Scaling Group, terminate EC2 instances, delete Aurora database cluster, remove ALB, delete NAT Gateway, and release Elastic IP addresses to avoid unexpected charges.

12. Results

The project successfully established a production-grade, highly available two-tier cloud architecture on AWS:

- **Multi-AZ Resilience:** Validated high availability and fault tolerance across us-east-1a and us -east-1b availability zones.
- **Seamless Load Balancing:** Successfully load balanced live client traffic from twotire.shop across backend targets.
- **Data Tier Security:** Backend Aurora MySQL database cluster completely protected within private subnets with internet ingress disabled.
- **Elastic Compute Capacity:** Automated provisioning established using custom AMIs, Launch


Edit with WPS Office

Templates, and Auto Scaling.

13. Future Improvements

- **Infrastructure as Code (IaC):** Codify entire VPC, compute, and database infrastructure using Terraform or AWS CloudFormation.
- **CI/CD Pipeline:** Establish automated build and deployment pipelines using GitHub Actions to update EC2 web application code.
- **Edge Security with AWS WAF:** Deploy CloudFront CDN caching and attach AWS WAF Web ACL rules for edge DDoS and OWASP Top 10 mitigation.
- **Automated CloudWatch Alarming:** Configure CloudWatch alarms with SNS email alerts for CPU utilization, target latency, and 5xx errors.

14. Conclusion

This project demonstrates practical mastery of production AWS cloud architecture. By deploying a multi-AZ two-tier architecture incorporating VPC subnet isolation, security group chaining, Application Load Balancing, Auto Scaling Groups, Amazon Aurora MySQL multi-AZ clusters, and Route 53 domain delegation, the infrastructure satisfies the highest industry standards for reliability, security, and scalability.

15. References

- **AWS Well-Architected Framework:** https://aws.amazon.com/architecture/well-architected/
- **Amazon VPC User Guide:** https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html
- **Elastic Load Balancing Documentation:** https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
- **Amazon Aurora MySQL User Guide:** https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html
- **Amazon Route 53 Developer Guide:** https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/Welcome.html
- **AWS Certificate Manager Guide:** https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html


Edit with WPS Office